

System Design — Interview Answers

Easy · Medium · Hard — Professional Answers with Real-World FANG Examples

Professional Answers · Real-World FANG Examples · All Difficulty Levels

EASY — 25 Questions with Answers

Q1. What is horizontal vs vertical scaling?

Vertical scaling (scale up): add more resources to an existing server — more CPU, RAM, faster disk. Simple: no code changes needed. Limits: there is a physical maximum (the largest machine available), it requires downtime to upgrade, and a single machine is a single point of failure. Horizontal scaling (scale out): add more servers and distribute the load across them. No theoretical limit on scale. Requires a load balancer and applications must be designed to be stateless. Cloud-native systems are built for horizontal scaling because commodity hardware is cheaper than enterprise hardware and individual server failures are tolerated.

Real-World Example:

Netflix handles 250M+ streaming sessions simultaneously through horizontal scaling. Their video encoding service spins up thousands of EC2 instances in parallel during peak demand (Friday evenings), each encoding a different quality level of a different video. A single vertically-scaled server could not encode 1,000 videos simultaneously regardless of its size. Netflix uses auto-scaling groups that monitor CPU utilisation — if average CPU > 70%, add instances; if < 30%, remove instances. This elasticity allows them to process the same 24-hour workload while running 10× fewer instances at 3am vs 8pm.

Q2. What is a load balancer? What are the main load balancing algorithms?

A load balancer distributes incoming requests across multiple backend servers to prevent any single server from being overwhelmed. It also performs health checks — removing unhealthy servers from rotation automatically. Main algorithms: Round Robin — each request goes to the next server in rotation sequentially. Least Connections — send to the server with the fewest active connections (better for variable-length requests). IP Hash — hash the client IP to always route the same client to the same server (useful for session affinity). Weighted Round Robin — servers get traffic proportional to their capacity weight. Random — choose a server randomly (effective with many healthy servers due to the birthday paradox).

Real-World Example:

Google's load balancer (Google Front End, GFE) uses a weighted round-robin algorithm that accounts for server health, geographic proximity, and current server load. When a server's CPU exceeds 80%, GFE dynamically reduces its weight in the rotation. GFE processes over 8.5 billion search queries per day across thousands of backend servers globally. During the Super Bowl, real-time queries spike 3× above baseline within seconds — GFE absorbs this by pre-warming extra capacity and adjusting weights instantly. Traditional DNS-based load balancing (round-robin DNS) cannot adapt quickly enough for such spikes because DNS TTLs prevent instant re-routing.

Q3. What is a CDN (Content Delivery Network)?

A CDN is a geographically distributed network of edge servers that cache and deliver content from locations physically close to end users. Instead of every request travelling to the origin server (potentially on the other side of the world), requests are served from the nearest edge node. Benefits: reduced latency (data travels fewer hops), reduced origin server load (cache absorbs repeat requests), better availability (content served even if origin is down), and improved performance for static assets (images, JS, CSS, videos).

Real-World Example:

Netflix's Open Connect CDN has 17,000+ servers in 6,000+ locations worldwide. During a popular show premiere (e.g. Stranger Things season launch), 60-70% of Netflix traffic is served directly from Open Connect servers in ISP data centres within customers' own networks, without ever leaving the local network. This reduces Netflix's bandwidth costs by billions of dollars per year and improves video start time from 8 seconds (without CDN) to under 2 seconds. A user in Mumbai watching a show stored on an Open Connect server in their ISP's Mumbai data centre experiences the same quality as a user in Silicon Valley watching from Netflix's origin.

Q4. What is a database index and why is it important?

A database index is a separate data structure (typically a B-tree) that enables the database to find rows matching a WHERE clause without scanning every row in the table. Without an index: the database reads every row (full table scan) — $O(n)$. With an index on the queried column: the database navigates the B-tree — $O(\log n)$. Indexes trade storage space and write speed for dramatically faster reads. Every indexed column requires updating the B-tree on every INSERT, UPDATE, and DELETE — indexes slow writes by 10-30%.

Real-World Example:

At Amazon, adding an index on the orders table's customer_id column transformed a query that took 45 seconds (full scan of 1B rows) to 8 milliseconds (index lookup + 200 rows fetched). The index saved Amazon from a production incident where the 'Your Orders' page was timing out for users with many orders. However, Amazon's database team also learned the cost: a batch ORDER UPDATE operation that previously ran in 2 minutes now took 18 minutes because of index maintenance overhead on 3 indexes per row updated. They added index conditionally and created a no-index batch update path for the bulk operations.

Q5. What is the CAP theorem?

The CAP theorem states that a distributed data store can provide at most 2 of these 3 guarantees simultaneously: Consistency (C) — every read returns the most recent write or an error. Availability (A) — every request receives a (non-error) response, though it may not contain the most recent data. Partition Tolerance (P) — the system continues operating even when some nodes cannot communicate (network partitions). Since network partitions occur in any distributed system and cannot be prevented, you must choose between CP (consistency + partition tolerance, accepting possible unavailability) or AP (availability + partition tolerance, accepting possible stale reads) when partitions occur.

Real-World Example:

Amazon DynamoDB is an AP system — it prioritises availability over consistency. When a network partition occurs, DynamoDB continues accepting reads and writes using the available replica, potentially returning stale data. This is correct for Amazon's shopping cart: a user who adds an item should see the cart update even if there is a partition. HBase (used at Facebook) is CP — it rejects reads/writes if it cannot confirm consistency, ensuring users always see accurate friend counts. The trade-off: DynamoDB's always-available AP model served 3 trillion requests on Prime Day 2023; any CP model would have returned errors during the network blips that occur during such extreme traffic.

Q6. What is eventual consistency? When is it acceptable?

Eventual consistency is a consistency model where distributed replicas are guaranteed to converge to the same value eventually — but not immediately. Between the moment a write is made and the time all replicas have received it, different clients may see different values. This is acceptable when: (1) slight staleness is tolerable (social media like counts, view counts), (2) high availability is more important than perfect accuracy (shopping carts, user preferences), (3) the system can reconcile conflicting writes (collaborative editing), (4) business logic can handle temporary inconsistencies (bank balance shown slightly differently on two devices).

Real-World Example:

Twitter's tweet like count uses eventual consistency — if you like a tweet in New York, a user in Tokyo may see the old count for a few seconds. This is perfectly acceptable: there is no business harm from temporarily showing 10,237 likes instead of 10,238. Twitter's alternative — strongly consistent like counts — would require locking a row during every like operation, serialising all writes to the same tweet, and causing timeouts during viral tweets receiving millions of simultaneous likes. The Netflix film 'Bird Box' received 45M views in its first week; its view count was eventually consistent — updated in batch every few minutes rather than transactionally.

Q7. What is a message queue? Give two use cases.

A message queue is a durable, ordered buffer that decouples message producers (senders) from consumers (receivers). Producers publish messages without waiting for consumers to process them. Consumers read messages at their own pace. Key properties: durability (messages survive server restarts), ordering (typically FIFO), at-least-once delivery (messages retried until acknowledged). Popular implementations: RabbitMQ, Apache Kafka, AWS SQS, Google Pub/Sub.

Real-World Example:

Use case 1 — Uber's ride request processing: when a rider requests a ride, the request is placed in a queue (SQS). Multiple consumer services read from the queue independently: the dispatch service assigns a driver, the notification service sends a push notification, the surge service updates pricing, the analytics service records the event. Without a queue, all these operations would be synchronous and a slow notification service would delay dispatch. Use case 2 — Facebook's comment processing: when a user comments on a post, the comment is published to Kafka. Consumers: content moderation service, notification service (notify the post author), search indexing service, and analytics service all consume independently. A message queue allows Facebook to add new consumers without changing the comment submission code.

Q8. What is the difference between SQL and NoSQL databases?

SQL (relational) databases store data in structured tables with predefined schemas, enforce relationships via foreign keys, support ACID transactions, and use SQL as a query language. Best for: complex queries with many joins, strong consistency requirements, financial transactions, structured data with known schema. NoSQL databases trade some SQL features for scalability and flexibility. Types: document (MongoDB), key-value (Redis, DynamoDB), wide-column (Cassandra), graph (Neo4j). Best for: flexible or evolving schemas, horizontal scalability, high write throughput, hierarchical or graph data.

Real-World Example:

At Instagram, user profiles are stored in PostgreSQL (SQL) — fixed schema, strong consistency needed for authentication, complex profile queries. User activity logs (every like, view, scroll) are stored in Cassandra (NoSQL wide-column) — 10B+ daily events, horizontal scaling across 100 nodes, high write throughput. Comment threads use Cassandra's time-series model (row key = post_id, column key = timestamp) for fast retrieval by post. Financial transactions (ads billing) use PostgreSQL — ACID required. Instagram's hybrid approach is typical: SQL for core entity data, NoSQL for high-volume event data.

Q9. What is caching? What cache eviction policies exist?

Caching stores frequently accessed data in fast memory (RAM) to avoid repeatedly fetching it from slower storage (disk, database, network). Cache hit: data found in cache — fast. Cache miss: data not in cache — fetch from source, store in cache. Hit ratio = hits / (hits + misses). Eviction policies determine what to remove when the cache is full: LRU (Least Recently Used) — remove the item not accessed for the longest time (most common, good for temporal locality). LFU (Least Frequently Used) — remove the item accessed the fewest times. FIFO — remove the oldest inserted item. TTL-based — remove items after a fixed time period regardless of access pattern.

Real-World Example:

At Facebook, Memcached caches user profile data with an LRU eviction policy. Active users' profiles stay in cache; inactive users' profiles are evicted. Cache hit rate: 99%+. Facebook originally used a simple LRU but found that viral content caused 'cache stampedes' — millions of simultaneous misses for the same content when the TTL expired. Fix: probabilistic early expiration (randomly refresh the cache slightly before TTL expiry to prevent simultaneous misses) and 'lease' mechanism (only one request fetches the source while others wait for the result). These two optimisations reduced database load spikes by 90% during viral content events.

Q10. What is a REST API? What are the key HTTP methods?

REST (Representational State Transfer) is an architectural style for APIs built on HTTP. Resources are identified by URLs. Stateless: each request contains all information needed; no session state on the server. Key HTTP methods: GET — retrieve a resource, no side effects, idempotent, cacheable. POST — create a new resource or submit data, not idempotent. PUT — replace a resource entirely, idempotent. PATCH — partial update of a resource. DELETE — remove a resource, idempotent. Status codes communicate results: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.

Real-World Example:

At Stripe, every payment operation is a REST API call. POST /v1/charges creates a new charge — not idempotent (calling it twice charges twice). Stripe solves this with idempotency keys: the client provides a unique key, and Stripe stores the result so duplicate POSTs return the same result without creating a second charge. GET /v1/charges/ch_123 retrieves a charge — safe to retry. DELETE /v1/subscriptions/sub_456 cancels a subscription — idempotent (calling it twice is safe). Stripe's REST API processes 250M+ API calls per day with 99.999% uptime. The stateless design means any Stripe server can handle any request — load balancers route requests freely without session affinity.

Q11. What is database sharding?

Sharding divides a large database table horizontally across multiple servers (shards). Each shard contains a subset of rows determined by a shard key. Common strategies: range sharding (user IDs 1-1M on shard 1, 1M-2M on shard 2 — simple but creates hot spots if recent IDs are most active), hash sharding (hash(user_id) % N determines the shard — even distribution), directory sharding (a lookup table maps keys to shards — flexible but introduces a lookup step). Benefits: queries involving only one user hit one shard (reducing load per shard). Costs: cross-shard queries and joins are expensive; resharding as you grow is complex.

Real-World Example:

Pinterest shards their pins database by user_id using consistent hashing. When you load your Pinterest profile, all your pins are on one shard — no cross-shard query needed. When you search for pins about 'recipe', the query fans out across all shards and results are merged — a scatter-gather query. Pinterest grew from 1 to 8 shards in their first year as user data grew. Consistent hashing minimised data movement during resharding: adding a 9th shard required moving only 1/9th of the data (from the most loaded shard to the new one) instead of re-hashing all data. Without sharding, a single MySQL server would not support Pinterest's 450M monthly users.

Q12. What is a microservices architecture? How does it differ from monolithic?

A monolith is a single deployable unit containing all application functionality — simple to develop initially, easy to test, but becomes hard to scale and deploy as it grows. A microservices architecture decomposes the application into small, independently deployable services that communicate over APIs (REST, gRPC, or message queues). Each service owns its own database, has its own deployment pipeline, and can be scaled independently. Benefits: independent scaling (scale only the bottleneck service), independent deployment (update one service without touching others), technology flexibility (different services can use different languages). Costs: distributed systems complexity, network latency between services, service discovery, distributed tracing.

Real-World Example:

Amazon decomposed their monolith into 1,000+ microservices. The product page alone calls 200+ services: pricing service, inventory service, review service, recommendation service, shipping estimate service, etc. Each can be scaled independently — the review service scales up during review campaigns without scaling the pricing service. Netflix uses 1,000+ microservices for the same reason. The cost of decomposition: Netflix's 2012 'Chaos Monkey' was built specifically because microservices failures cascade in unexpected ways — they introduced intentional failures to build resilience. Amazon estimated the monolith's deployment took 12 hours; individual microservice deployment takes 10 minutes and affects only 1 of 1,000+ services.

Q13. What is a reverse proxy? How does it differ from a forward proxy?

A forward proxy sits between clients and the internet, forwarding client requests to servers. The server sees the proxy's IP, not the client's — provides client anonymity, content filtering, corporate access control. A reverse proxy sits between the internet and backend servers. Clients talk to the reverse proxy, which forwards to the appropriate backend server. The client may not know how many backend servers exist. Benefits of reverse proxy: load balancing, SSL termination (decrypt HTTPS once at the proxy, use HTTP internally), caching, compression, request routing, security (backend servers not exposed directly to the internet).

Real-World Example:

Nginx is the most widely deployed reverse proxy — used by 30%+ of all websites. At Cloudflare, every website behind Cloudflare's protection routes through Cloudflare's reverse proxy infrastructure. The reverse proxy terminates TLS (decrypts the HTTPS connection), caches cacheable responses (reducing origin server load by 60-80%), and applies DDoS protection rules before forwarding legitimate traffic to origin servers. Netflix's API gateway (Zuul) acts as a reverse proxy for all microservice calls — routing /api/users requests to the user service and /api/recommendations to the recommendation service. Without the reverse proxy, clients would need to know the address of every microservice.

Q14. What is a circuit breaker pattern?

A circuit breaker prevents a system from repeatedly calling a failing service, giving it time to recover. Three states: Closed (normal) — requests flow through; if failures exceed a threshold, trip to Open. Open (failing) — all requests immediately fail (fast fail) without calling the downstream service, for a timeout period. Half-Open (testing recovery) — after the timeout, allow one test request. If it succeeds: close the circuit. If it fails: return to Open. This pattern prevents cascading failures: a slow payment service should not cause the entire shopping cart to hang, consuming threads waiting for timeouts.

Real-World Example:

Netflix's Hystrix library (now Resilience4j) implements circuit breakers for all inter-service calls. During a 2012 AWS outage, the recommendation service became slow. Without circuit breakers, all user requests were hanging waiting for recommendations — the whole homepage was unresponsive. With Hystrix: the circuit breaker detected >50% failure rate, opened the circuit, and the homepage immediately rendered with cached or default recommendations instead of hanging. The homepage remained available (degraded gracefully) rather than becoming completely unresponsive. Netflix's design principle: every service call has a circuit breaker with a fallback — 'never let a downstream failure propagate to the user.'

Q15. What is a primary-replica (master-slave) database setup?

A primary-replica setup has one primary (master) server that handles all write operations, and one or more replica (slave) servers that receive copies of the primary's changes asynchronously (or synchronously). The primary applies the write and logs it; replicas apply the same log entries. Reads can be distributed to replicas — horizontal scaling of read capacity. If the primary fails, a replica is promoted to primary (failover). Benefits: read scaling, high availability, geographic distribution. Costs: replication lag (replicas may be seconds behind — eventual consistency), more complex failover, replicas have slightly outdated data.

Real-World Example:

At Stack Overflow, their PostgreSQL setup has 1 primary and 11 read replicas. Their traffic is 90% reads (people reading answers) and 10% writes (posting answers). All reads are distributed across 11 replicas — 11× the read capacity of a single server. The single primary handles all writes. During the 2020 COVID lockdowns, Stack Overflow traffic doubled. The replica scaling absorbed the increase without changing the primary. Replication lag averaged 5ms — acceptable for a Q&A; platform where reading a 2-second-old answer is fine. Stack Overflow serves 60M unique monthly visitors from this PostgreSQL setup — a remarkably simple infrastructure for the scale.

Q16. What is a CDN cache invalidation and why is it hard?

Cache invalidation is the process of removing or updating cached content across CDN edge servers when the origin content changes. It is hard because: CDN edge servers are globally distributed (thousands of nodes), and propagating a cache purge to every node takes time (seconds to minutes). During this window, different users in different locations may see different versions of the content. Methods: TTL-based expiry (content automatically expires after N seconds — simple but creates stale content windows). Active purge (explicitly invalidate content by URL when it changes — faster but requires API calls to CDN). Versioned URLs (embed a hash or version in the URL: logo.v2.png — old URL remains valid, new URL automatically misses cache — instant effective invalidation).

Real-World Example:

Facebook uses versioned URLs for all static assets: every CSS file, JavaScript bundle, and image gets a URL containing a content hash (e.g. main.abc123.js). When a new JS bundle is deployed, the URL changes — all CDN nodes immediately serve the new file without any purge needed (the new URL is a cache miss everywhere, fetching fresh from origin). The old URL remains in CDN cache until TTL expires, but no user will request it (no HTML links to it). This strategy makes cache invalidation trivial: you never need to purge, because you never update content at the same URL. Facebook serves 6M+ static asset requests per second using this strategy with no cache invalidation infrastructure needed.

Q17. What is the difference between synchronous and asynchronous processing?

Synchronous processing: the caller waits (blocks) until the operation completes before proceeding. Simple to code and reason about. If the operation is slow, the caller is stuck waiting — this can cascade if the caller is itself a server handling requests. Asynchronous processing: the caller initiates the operation and continues without waiting. The result is delivered later via callback, event, promise, or message. More complex but allows the caller to continue doing work. Enables non-blocking I/O — a single thread can manage thousands of concurrent connections.

Real-World Example:

At Stripe, payment processing has both components. Synchronous: the API call validates the card number, checks for fraud (sub-100ms), and returns a 200 OK immediately — the merchant's website must wait for this. Asynchronous: sending the payment receipt email, updating analytics dashboards, and notifying bank systems happen asynchronously via a message queue. The merchant's checkout form completes in <300ms because only the critical path is synchronous; the receipt email might arrive 10 seconds later. If the email service goes down, payments still succeed — the email is retried from the queue. Mixing sync and async allows Stripe to offer a fast, reliable checkout experience while maintaining eventual consistency for non-critical downstream operations.

Q18. What is service discovery?

Service discovery is the mechanism by which services in a microservices architecture find each other's network addresses (IPs and ports). In cloud environments, services spin up and down dynamically — a payment service might run on a different IP every time it's deployed. Static IP configuration fails. Two models: client-side discovery — clients query a service registry (Consul, Eureka, Zookeeper) to find the service address, then call it directly. Server-side discovery — clients call a load balancer or API gateway, which queries the registry and forwards the request.

Real-World Example:

At Netflix, every microservice registers itself with the Eureka service registry on startup and sends heartbeats every 30 seconds. If a service fails to heartbeat, Eureka removes it after 90 seconds. When the recommendation service needs to call the user profile service, it asks Eureka: 'Where are the user profile service instances?' Eureka returns a list of healthy instances (typically 5-10 IPs). The recommendation service's client-side load balancer (Ribbon) picks one and makes the call. Without Eureka, Netflix would need manually-updated configuration files for 1,000+ services × thousands of instances — impossible to maintain in a cloud environment where instances come and go every minute.

Q19. What is rate limiting? Name two algorithms.

Rate limiting controls the number of requests a client can make within a time window, protecting the system from overload and abuse. Algorithm 1 — Token Bucket: each client has a bucket with capacity C tokens. Tokens refill at rate R per second. Each request consumes one token. Allow if tokens > 0; reject if empty. Allows bursting up to C requests, then enforces the sustained rate R. Algorithm 2 — Sliding Window Counter: divide time into small buckets (e.g. 1-second buckets). Count requests in the last N seconds by summing the last N buckets. Reject if count exceeds the limit. More accurate than fixed windows which allow 2× the limit at window boundaries.

Real-World Example:

GitHub uses rate limiting on their API: authenticated requests are limited to 5,000 per hour per token (token bucket with 5,000 capacity, refills at ~1.4 tokens/second). This prevents a single bad script from hammering their API and degrading performance for all 100M+ developers. GitHub returns rate limit headers with every response: X-RateLimit-Remaining, X-RateLimit-Reset — so clients can back off gracefully before hitting the limit. When a developer's CI/CD pipeline accidentally made 10,000 API calls in 5 minutes (infinite retry loop), rate limiting returned 429 Too Many Requests and stopped the runaway process rather than allowing it to overwhelm GitHub's API servers.

Q20. What is the difference between latency and throughput?

Latency is the time it takes to complete one operation — the delay from request to response. Measured in milliseconds or seconds. Google targets sub-100ms for search results because human perception notices delays above 100ms. Throughput is the number of operations completed per unit of time — requests per second (RPS) or queries per second (QPS). A system can have low latency AND high throughput, or they can trade off: you can increase throughput by batching requests (groups requests together) which increases individual request latency. Optimising for one doesn't automatically improve the other.

Real-World Example:

AWS Lambda illustrates the latency-throughput trade-off. A single Lambda invocation has p99 latency of 10-100ms (fast for one request). Maximum throughput: 1,000 concurrent invocations per region by default (10,000 with increased limits). At peak, AWS Lambda handles millions of invocations per second globally — extremely high throughput. However, 'cold start' latency (when a new Lambda instance must be initialised) can be 500ms-2s — high latency for the first request. DynamoDB is tuned for high throughput (millions of requests/second) AND low latency (single-digit milliseconds) — but achieves this by trading consistency (eventual consistency) and complex queries (no joins).

Q21. What is a data pipeline?

A data pipeline is a series of data processing steps that move and transform data from source to destination. Components: source (databases, APIs, event streams, files), processing (transformation, cleaning, aggregation, enrichment), and destination (data warehouse, analytics system, ML training data). Types: batch pipeline (processes data in large chunks at intervals — nightly ETL jobs), streaming pipeline (processes data continuously as it arrives — real-time dashboards). Tools: Apache Spark (batch), Apache Kafka + Flink (streaming), Airflow (workflow orchestration), dbt (SQL transformations in data warehouse).

Real-World Example:

At Uber, their streaming data pipeline processes 1 trillion events per day. The pipeline: ride requests, GPS pings, and fare calculations flow into Kafka (streaming ingestion). Apache Flink consumers process the Kafka stream in real time to compute metrics (surge pricing, ETA estimates). The processed data flows into Apache Pinot (real-time OLAP database) for operational dashboards. Separately, a batch pipeline copies the data from Kafka to HDFS, runs Spark transformations to compute daily reports, and loads results into Presto for analytical queries. Uber uses Apache Airflow to orchestrate the batch pipeline — a DAG of tasks with dependencies, scheduling, and failure recovery.

Q22. What is a distributed transaction? What is 2PC?

A distributed transaction spans multiple services or databases. It must maintain ACID properties across all participants — either all commit or all roll back. Two-Phase Commit (2PC): Phase 1 (Prepare) — the coordinator asks all participants: 'Are you ready to commit?' Each participant writes to its local log and responds Yes or No. Phase 2 (Commit/Rollback) — if all said Yes: coordinator sends Commit to all; if any said No: sends Rollback to all. Problem: if the coordinator crashes after Phase 1 but before Phase 2, participants are in a blocking state (they prepared but don't know the outcome) — the system is stuck until the coordinator recovers.

Real-World Example:

Stripe avoids distributed transactions using the Saga pattern instead of 2PC. When processing a payment that involves charging the customer (Stripe service), paying the merchant (bank transfer service), and updating inventory (merchant service), Stripe uses a choreography-based saga: each service completes its operation and emits an event. If any step fails, compensating transactions undo previous steps. For a failed payment: the charge is reversed (compensating transaction) and the merchant is notified. 2PC would require Stripe's payment service to hold locks while coordinating with the merchant's system — unacceptable because Stripe cannot guarantee the merchant's system will respond within a reasonable time, blocking the payment database.

Q23. What is a bloom filter?

A Bloom filter is a probabilistic data structure that tests whether an element is a member of a set. Space-efficient: uses k hash functions and a bit array. Add(x): hash x with all k functions, set those bit positions to 1. Check(x): hash x with all k functions, if ALL bit positions are 1 — probably in the set (false positive possible). If ANY bit position is 0 — definitely not in the set (no false negatives). False positive rate is tunable by choosing the bit array size and number of hash functions. Cannot delete elements (unless using Counting Bloom Filter). Extremely space-efficient: storing 1M elements with 1% false positive rate requires only ~1MB.

Real-World Example:

At Google's Bigtable, Bloom filters are used to avoid unnecessary disk lookups. When a read request comes in, Bigtable first checks a Bloom filter to determine if the key might exist in an SSTable (disk file). If the filter says 'definitely not there' (zero false negatives): skip the disk read — a huge savings at petabyte scale. If the filter says 'might be there': do the disk read. With a 1% false positive rate, 99% of negative lookups are answered without any disk I/O. Disk reads are 1,000× slower than memory reads. Chrome uses Bloom filters to check URLs against a list of malicious sites — the filter is downloaded to the browser (very small) and most URLs are checked locally without any network request.

Q24. What is a service mesh?

A service mesh is an infrastructure layer that handles service-to-service communication in a microservices architecture. It adds a sidecar proxy (e.g. Envoy) to every service instance that intercepts all inbound and outbound traffic. The service mesh provides: mutual TLS (mTLS) encryption between services, automatic retries and circuit breaking, service discovery, load balancing, distributed tracing, and metrics collection — all without code changes to the services themselves. Popular service meshes: Istio (uses Envoy sidecar), Linkerd, Consul Connect.

Real-World Example:

At Lyft, which created Envoy, the service mesh solved the 'polyglot observability' problem. With 500+ microservices in Python, Go, Java, and C++, implementing circuit breaking and distributed tracing in each service's code in every language was impractical. The sidecar proxy approach: deploy Envoy alongside every service; configure retries, timeouts, circuit breaking centrally in Istio. When a Lyft service starts throwing errors, Envoy automatically circuit-breaks without any service code change. Distributed traces (Zipkin/Jaeger) show the exact latency breakdown across all service calls for every user request — enabling engineers to identify the bottleneck in a 20-hop request chain within seconds.

Q25. What is a write-ahead log (WAL)?

A Write-Ahead Log (WAL) is a durability mechanism where every database modification is first written to an append-only log file BEFORE it is applied to the actual data files. The log entry is written to disk first (fsync), then the data file is modified. If the system crashes after the WAL entry but before the data file update, the WAL is replayed on recovery to redo the operation. This ensures no committed transaction is ever lost, even if the server crashes mid-write. WAL also enables replication: replicas receive and apply the WAL stream rather than copying the full data files.

Real-World Example:

PostgreSQL's WAL (also called the transaction log) is the foundation of its crash recovery and streaming replication. Every INSERT, UPDATE, DELETE is first written to WAL (sequential write — fast), then applied to heap files (random write — slower). If PostgreSQL crashes, the WAL is replayed from the last checkpoint on restart — recovery in seconds. For replication: the WAL stream is sent from primary to replicas in real-time. Replicas apply WAL entries to maintain a replica of the primary. Amazon RDS for PostgreSQL uses this WAL stream to maintain read replicas across availability zones. The WAL's sequential write pattern (no random seeks) is why PostgreSQL can sustain 100K+ writes/second on SSDs — sequential writes are 10-100× faster than random writes.

MEDIUM — 10 Questions with Answers

Q1. Design a URL shortening service like bit.ly.

Functional requirements: shorten URL → short code, redirect short code → original URL. Non-functional: 100M URLs, <10ms redirect latency, high availability. Short code generation: encode the unique auto-incremented ID in base62 (a-z, A-Z, 0-9). 7 characters of base62 = $62^7 \approx 3.5$ trillion unique URLs — sufficient. Database: URLs table (short_code PK, original_url, created_at, user_id, click_count). Storage estimate: $100M \times 500$ bytes avg = 50GB — fits in a single DB but use read replicas for scale. Caching: Redis with LRU for top 20% of short codes that handle 80% of traffic (Pareto). Cache on redirect: short_code → original_url, TTL=24h. Redirect flow: receive short_code → check Redis → if miss, check DB → 301 (permanent) or 302 (temporary, preserves click analytics) redirect. Unique code generation at scale: use a distributed ID generator (Snowflake ID) to avoid collisions without coordination.

Real-World Example:

Bit.ly processes 600M+ redirects per day (7,000 req/s average, 50,000 req/s peak). Their architecture: Nginx reverse proxy → Go microservices → Redis cluster (cache) → Cassandra (persistent storage). Short codes are 6 characters of base62. Redis cache stores the most-clicked 50M codes — cache hit rate >90%. Cassandra stores all 40B+ historical URLs partitioned by short_code for $O(1)$ lookups. Redirect latency: p50=5ms, p99=25ms. The key design choice: 302 redirects (temporary) instead of 301 (permanent) allow Bit.ly to track every click — 301 redirects are cached by browsers, bypassing Bit.ly's servers entirely. Custom domains (brand.link/code) are handled by Nginx routing rules that map domains to Bit.ly customer accounts.

Q2. Design a rate limiter for an API serving 10,000 requests/second.

Requirements: per-user 100 req/sec limit, global 10,000 req/sec limit, <1ms overhead per request, distributed (multiple API servers). Algorithm: Token bucket with Redis Lua script (atomic operations). Per request: EVAL lua_script(key=user_id, capacity=100, rate=100, now=timestamp). Lua script: get (tokens, last_time), compute elapsed time, compute new tokens = min(capacity, stored + elapsed × rate). If tokens >= 1: set tokens-1, return ALLOW. Else: return DENY. Single Redis round trip (Lua is atomic). Latency: Redis RTT ~0.3ms. Sliding window counter alternative: Redis sorted set per user (ZADD + ZCOUNT + ZREMRANGEBYSCORE). More accurate at window boundaries but 3 Redis operations per request. Architecture: API servers → Redis cluster (16 shards by user_id hash). Rate limit headers in response: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset.

Real-World Example:

GitHub's API rate limiter limits authenticated users to 5,000 requests/hour. Implementation: a Lua script in Redis performs atomic token bucket operations. The Lua script runs on the Redis server — no network round trips between read and write. Without atomicity (GET then SET separately), two concurrent requests from the same user could both see tokens=1, both decrement, and both succeed — allowing 2 requests when only 1 should succeed. GitHub's Redis cluster has 8 shards; user tokens are stored on the shard determined by hash(user_id). With 100M API users and 10 bytes per user state (8 for token count float + 2 for timestamp delta), the rate limiter state is 1GB — easily fits in Redis. GitHub returns 429 Too Many Requests with Retry-After header when limits are exceeded.

Q3. Design a system for distributed caching.

Requirements: 1TB total cache, sub-millisecond latency, high availability, auto-scaling. Architecture: consistent hashing ring with N cache nodes. Key assignment: $\text{hash}(\text{key}) \rightarrow \text{position on ring} \rightarrow \text{serve the next clockwise node}$. Consistent hashing ensures only k/N keys are remapped when a node is added/removed (vs k keys for modulo hashing). Each cache node stores its shard of the keyspace. Clients use a client library that maintains the consistent hash ring and routes directly to the correct cache node — no single point of failure proxy. Replication: each key is replicated to $R=2$ successor nodes on the ring for fault tolerance. On read: try the primary node; on cache miss, try replicas. On write: write to all R replicas. Eviction: LRU per node. Hot key mitigation: replicate hot keys to more nodes (local replication factor); detect hot keys via frequency sampling.

Real-World Example:

Twitter's Twemproxy (now Redis Cluster) uses consistent hashing across their 1,000+ Redis cache nodes. Each node holds ~50-100GB of cache data (user timelines, tweet text, user follower counts). Consistent hashing ensures that when they add 50 more nodes to handle growth, only ~4.5% of keys need to be remapped (vs 100% with modulo hashing). Twitter's Redis cluster serves 1M+ requests/second per cluster. Hot key problem: Elon Musk's profile data is accessed billions of times/day — their solution is 'multilayer caching': L1 = in-process cache on each API server (fast, no network hop), L2 = Redis cluster. The in-process L1 cache absorbs the hot key traffic without overloading any single Redis node.

Q4. Design a notification system (email, SMS, push).

Components: (1) Notification Service API — accepts notification requests (user_id, type, message, priority). (2) Worker queues — separate Kafka topics per channel (email_queue, sms_queue, push_queue) and priority (high, normal). Priority segregation ensures transactional alerts (payment failed) don't queue behind marketing emails. (3) Channel workers — email (SendGrid/SES), SMS (Twilio), push (FCM/APNS). Workers process from queues, handle retries with exponential backoff. (4) User preference service — lookup user's notification preferences and channel opt-outs before sending. (5) Deduplication — idempotency key per notification; Redis set tracks sent notification IDs for 24h to prevent duplicates on retry. (6) Observability — track delivery rate, bounce rate, open rate per template and channel.

Real-World Example:

At Airbnb, the notification system sends 200M+ notifications per day (booking confirmations, payment receipts, host messages). Architecture: booking events → Kafka → Notification Service → 3 separate Kafka topics (email, SMS, push) → channel workers. Transactional emails (booking confirmation) use a high-priority queue — processed in <5 seconds. Marketing emails use a separate low-priority queue — can lag by hours. Twilio sends 10M SMS/day; rate limiting at Twilio requires the workers to respect 100 SMS/second per long code. User preference service: queried before every notification — users who opted out of SMS never receive an SMS. Deduplication: a Redis set stores notification IDs for 24h — a Kafka consumer retry won't re-send the same booking confirmation twice.

Q5. Design a key-value store from scratch.

Storage engine: Log-structured merge tree (LSM tree). Writes: append to an in-memory buffer (MemTable, sorted by key). When MemTable exceeds 4MB: flush to an immutable Sorted String Table (SSTable) on disk. SSTables are periodically compacted (merged and deduplicated). Reads: check MemTable first → check Bloom filter for each SSTable (skip if definitely absent) → binary search the SSTables. WAL: before writing to MemTable, append to Write-Ahead Log for crash recovery. API: get(key) → value, put(key, value), delete(key) (tombstone marker). Performance: writes are sequential (fast, O(1) amortised), reads are O(log n) with Bloom filter acceleration. Replication: multi-Paxos or Raft consensus for fault-tolerant distributed key-value store.

Real-World Example:

Google's Bigtable (and Apache HBase) uses this exact LSM tree architecture. RocksDB (used by Meta, LinkedIn, Cockroach DB) is a production LSM key-value store embedded in many systems. Meta's RocksDB stores: ZippyDB (key-value store built on RocksDB) holds 10B+ key-value pairs for user data. The LSM tree's sequential writes are key to Meta's performance: SSDs degrade with random writes but sustain sequential writes at full throughput. The Bloom filter optimisation is critical: RocksDB's Bloom filter allows 99% of 'key not found' lookups to avoid any disk I/O. Without Bloom filters, every cache miss in RocksDB would require scanning multiple SSTables on disk — a 100× slower cold read path.

Q6. Design a typeahead/autocomplete system.

Requirements: real-time prefix search across 1B+ queries, <100ms latency, personalised. Architecture: (1) Trie stored in memory on search suggestion servers. Each node stores top-10 completions by frequency (precomputed). Trie size: $1M \times 10 \text{ char avg} \times 52 \text{ bytes/node} = \sim 500\text{MB}$ — fits in RAM. (2) For scale across multiple servers: partition the trie by first 2 characters of the prefix ($26^2 = 676$ partitions). (3) Offline aggregation: batch job (runs every 30 min) processes query logs, updates frequencies, regenerates the top-10 lists at each trie node. (4) Real-time trending: a stream processor monitors trending queries in the last 1 minute; these are injected into the trie with boosted frequency. (5) Personalisation: blend global top-10 with the user's own recent queries (stored in Redis, TTL=7 days).

Real-World Example:

Google Suggest (now Google Autocomplete) serves 8.5B+ queries/day with sub-100ms response. Their trie is distributed across thousands of servers worldwide with geographic caching. Each data centre maintains a full copy of the global trie (updated every 30 minutes). Real-time trending (e.g. 'earthquake san francisco' spiking after an event) is handled by a separate 'trending queries' stream that injects results into the suggestion layer within 60 seconds of the trend starting — much faster than the 30-minute batch update. Personalisation: if you've previously searched for 'Python' and type 'Py', Python appears first. The global trie answers in $O(\text{prefix_length}) = O(7)$ steps; personalisation lookup in Redis adds <2ms. Total latency: <30ms.

Q7. Design Instagram's photo storage system.

Requirements: 100M+ photos/day upload, global access, any-size retrieval, 99.999% durability. Upload flow: client → API server → validate → generate unique photo_id (timestamp + machine ID) → upload to object store → write metadata to DB → return URL. Storage: object storage (S3 equivalent) for raw photos + CDN for delivery. Database: photos table (photo_id, user_id, s3_key, created_at, metadata). Photo processing: async pipeline (Kafka) for generating thumbnail variants (thumbnail 150px, feed size 640px, full 1080px, etc.) — stored as separate objects with derived paths. CDN: photos served from the CDN closest to the user. Access control: public photos served via CDN directly; private photos via signed URLs (expire after 15 minutes).

Real-World Example:

Instagram stores 100M+ photos per day. Their actual stack: uploaded photos go to S3 (the original), then their image processing pipeline (Rust) generates 5 size variants asynchronously. Each variant is stored in S3 with a predictable path: {user_id}/{photo_id}/{size}.jpg. Facebook's CDN (Open Connect and PoP servers) caches and serves these at 100M requests/second globally. Metadata in Cassandra: user_id → list of photo_ids (wide row per user). Photo IDs are Instagram-style snowflake IDs encoding timestamp + shard ID + sequence — sortable by time, no central coordinator needed. When Instagram was acquired by Facebook in 2012, they had 13 employees, 30M users, and this architecture. The design was so efficient that Facebook barely changed it — they mainly added scale.

Q8. Design a distributed task scheduler.

Requirements: schedule tasks with arbitrary delays, execute at most once, resilient to failures. Architecture: (1) Task submission API: client submits (task_id, payload, execute_at). (2) Storage: tasks table in DB (task_id PK, payload, execute_at, status: PENDING/PROCESSING/DONE/FAILED, claimed_by, claimed_at). (3) Workers: continuously scan for due tasks (execute_at ≤ now() AND status = PENDING). Use SELECT FOR UPDATE SKIP LOCKED to claim tasks atomically — prevents two workers from claiming the same task. Set status = PROCESSING, claimed_by = worker_id, claimed_at = now(). Execute task. Set status = DONE. (4) Failure recovery: a sweeper process finds tasks where status = PROCESSING AND claimed_at < now() - 5 min — these indicate crashed workers. Reset to PENDING. (5) Scaling: partition tasks by (execute_at bucket, task_id mod N) for parallel scanning.

Real-World Example:

Sidekiq (Ruby) and Celery (Python) implement distributed task schedulers used by millions of applications. Stripe uses a custom distributed scheduler for: sending payment confirmation emails (delay 5 seconds to allow payment to settle), automatic subscription renewal charges (scheduled exactly at subscription renewal time), and failed payment retries (exponential backoff: retry after 1h, 2h, 4h, 8h). SELECT FOR UPDATE SKIP LOCKED is the key PostgreSQL feature that allows 100+ worker processes to claim tasks from the same queue without contention — each worker atomically claims a different task. Without SKIP LOCKED, workers would all lock on the same top-N tasks, causing contention.

Q9. Design a real-time leaderboard system.

Requirements: 10M+ users, update score in real-time, retrieve rank in $O(\log n)$, top-K leaderboard in $O(K)$. Data structure: Redis Sorted Set (ZSET) — $O(\log n)$ add/update score, $O(\log n)$ rank retrieval, $O(K)$ top-K retrieval. Commands: ZADD leaderboard user_id score ($O(\log n)$), ZRANK leaderboard user_id ($O(\log n)$ for rank), ZREVRANGE leaderboard 0 9 WITHSCORES ($O(K)$ for top K). For 10M users: ZSET uses ~200MB — fits in a single Redis node. For global scale: shard by game_id or region. Persistence: Redis RDB snapshots every 5 minutes; AOF for transaction log. Score update propagation: game server → Kafka → score update service → Redis ZADD. Near-real-time (<1 second delay) for leaderboard updates.

Real-World Example:

Zynga uses Redis Sorted Sets for leaderboards across FarmVille, Words with Friends, etc. With 250M registered users, per-game leaderboards use ZSET with game_id:period as the key. ZADD processes 100K score updates/second during peak gaming hours. ZRANK returns a player's rank in $O(\log n) = O(\log 10M) \approx 23$ operations — instantaneous. ZREVRANGE returns the top 100 players in $O(100) = \text{constant time}$. Zynga discovered a scaling limit: Redis Sorted Sets become memory-intensive above 100M entries (each entry ~50 bytes = 5GB per leaderboard). Solution: segment leaderboards by cohort (top 1M players in a dedicated ZSET; others in a separate approximate structure). Top players see exact rankings; others see approximate ranks.

Q10. Design a web crawler.

Requirements: crawl 1B web pages within 1 month, respect robots.txt, politeness delay. Architecture: (1) URL frontier: distributed priority queue (Kafka partitioned by domain hash). Priority = pagerank estimate / time_since_last_crawl. (2) DNS resolver: cache DNS lookups (TTL-aware) to avoid re-resolving the same domain per URL. (3) Fetcher service: HTTP fetcher with timeout (10s). Respect per-domain rate limits (1 request/2 seconds). (4) Parser: extract links from HTML (BeautifulSoup/Jsoup). Deduplication: Bloom filter for URLs seen (1B entries, 1% false positive = 1.2GB). (5) Content store: raw HTML to S3. URL + metadata to Cassandra. (6) Politeness: per-domain delay queue ensures no domain is crawled faster than 1 req/2 sec. (7) Monitoring: crawl rate (pages/sec), fresh content rate, URL queue depth per domain.

Real-World Example:

GoogleBot crawls 60+ trillion pages. Key architectural decisions: URL frontier uses a priority queue — high PageRank pages are crawled more frequently. Politeness: Google respects crawl-delay in robots.txt and never hits the same server faster than it can handle. Content deduplication: SimHash (near-duplicate detection) avoids storing near-identical pages (e.g. the same article with different ads). GoogleBot is distributed across hundreds of global crawlers; URL frontier is sharded by domain to ensure politeness (all GoogleBot requests to amazon.com go through one queue, rate-limited). DNS caching is critical: resolving DNS for every URL at 10B URLs/day would overwhelm DNS servers — Google caches DNS with TTL-aware invalidation. GoogleBot's politeness failures (crawling too aggressively) have historically caused website outages — politeness controls are first-class features.

HARD — 10 Questions with Answers

Q1. Design YouTube's video streaming system. Address upload, transcoding, storage, and delivery.

Upload: client uploads chunked (5MB chunks) to upload service → GCS blob storage. Each chunk is checksummed (MD5). On last chunk, upload service assembles the complete video. Transcoding: upload triggers an event → Kafka → transcoding job queue. Transcoding workers (GPU instances) produce 8 quality tiers: 144p, 240p, 360p, 480p, 720p, 1080p, 1440p, 4K. Each tier encoded with H.264 (compatibility) and VP9/AV1 (30% better compression). For a 10-minute 1080p source: produces 8 output files, ~2-3 minutes of transcoding on a modern GPU. Storage: source video and all tiers stored in GCS. Metadata in Spanner (video_id, title, uploader, tiers_available, thumbnails). Streaming: client player (adaptive bitrate — ABR) starts at a quality tier based on measured bandwidth, switches dynamically. Video segments: each quality tier split into 2-10 second MPEG-DASH or HLS segments, each a separate GCS object. CDN (YouTube's PoP network) caches popular video segments globally. Delivery: player requests segments from the nearest CDN PoP. Non-popular videos (the long tail — 95% of content, 20% of views) served from origin GCS.

Real-World Example:

YouTube processes 500 hours of video uploaded per minute. Transcoding pipeline: $500 \text{ hours} \times 60 \text{ min} \times 8 \text{ tiers} \times \sim 2\times \text{ real-time transcoding factor} = 480,000 \text{ GPU-hours per hour of real time}$ — a massive compute fleet. YouTube's ABR player switches quality every 2-10 seconds based on bandwidth. A user on a slow train connection might switch from 1080p to 360p mid-video — the player preloads 30 seconds of the current quality to buffer against quality switches. YouTube's CDN serves 1 billion+ streaming hours per day. The long-tail problem: most videos are watched infrequently — YouTube uses a tiered storage strategy where popular videos are on flash (low latency), less popular on spinning disk, and rarely-watched videos in cold storage (high latency retrieval, minutes to first byte).

Q2. Design a distributed message queue like Kafka.

Core concepts: Topic → ordered log of records. Each topic split into Partitions (parallel processing). Each partition replicated across $R=3$ Brokers. Producers write to the partition leader. Consumers read from any replica. Write path: producer → hash(key) → partition → leader broker → write to partition log (append only). Leader writes to local log, sends to followers. When $\min(\text{ISR replicas})$ acknowledge: send ack to producer. Read path: consumer group fetches from partition. Each partition assigned to exactly one consumer in a group. Consumer tracks offset (position in the log). On consumer restart: resume from stored offset. Ordering guarantee: within a partition, strict ordering. Across partitions: no ordering guarantee (use the same key to route related messages to the same partition). Retention: log segments deleted after TTL (7 days default) or size limit. Leader election: ZooKeeper (legacy) or KRaft (Kafka's own Raft implementation).

Real-World Example:

LinkedIn created Kafka in 2011 to handle 1B+ events per day. Today Kafka processes 7 trillion messages per day at LinkedIn across 4,000+ brokers. Key design decisions: the append-only log (sequential disk writes) is why Kafka can sustain millions of writes/second — sequential disk writes approach memory speeds on modern hardware. Consumer offset management: early Kafka stored offsets in ZooKeeper (bottleneck), moved to an internal `__consumer_offsets` topic (scales with Kafka itself). Exactly-once semantics (EOS): achieves it by combining idempotent producers (dedup by producer_id + sequence number) + transactional producers + consumer offset commits in the same transaction. This is critical for financial event streams where duplicate processing means charging customers twice.

Q3. Design a system to detect trending topics in real-time from Twitter's tweet stream.

Requirements: detect trending topics within 1 minute, global and per-country trends, 500K tweets/sec at peak. Architecture: Ingestion: tweets → Kafka (partitioned by tweet_id). Stream Processing: Flink or Spark Streaming reads from Kafka. Sliding window (1 minute, slide every 5 seconds): for each tweet, extract hashtags, mentions, n-grams. Count mentions using Count-Min Sketch (approximate, bounded memory: $d=5$ hash functions $\times w=100,000$ counters = 500,000 integers $\approx$ 2MB per time window). After each window: get top-1000 candidates from sketch using a parallel min-heap. Compare count to the previous window's count: trend score = $(\text{current_count} - \text{previous_count}) / \sqrt{\text{previous_count} + 1}$. This normalises for base popularity (prevents always-popular terms from dominating). Top-K selection: keep the top 50 trending topics per region. Output to Redis (key: region, value: sorted trending list). Frontend reads from Redis ($<1\text{ms}$).

Real-World Example:

Twitter's actual trending algorithm (Trending Happenings) runs on a stream processing cluster consuming the full tweet firehose. The core insight: trending is defined by a spike in mentions relative to baseline, not absolute volume. 'Justin Bieber' might have more daily mentions than 'earthquake' but 'earthquake' trends because its count spiked 1000% in 10 minutes (vs Bieber's steady baseline). Count-Min Sketch allows Twitter to track millions of n-grams simultaneously with a fixed memory budget — crucial since the number of distinct phrases in 500K tweets/second is unbounded. Twitter's trending algorithm also filters for quality: spam accounts (bot ring), duplicated tweets, and accounts with very low follower counts are down-weighted to prevent manipulation. The full algorithm involves 10+ signals beyond raw count.

Q4. Design a fraud detection system for financial transactions.

Architecture: two-tier decision engine. Tier 1 (real-time, $<50\text{ms}$): rule engine + lightweight ML model. Rules: velocity checks (>5 txn in 1 minute from same card), geographic anomalies (transactions in 2 countries within 1 hour), amount anomalies ($>3\sigma$ from 30-day average). ML: gradient boosting on 150 features, served via ONNX, 5ms inference. If score > 0.8 : auto-block and require 2FA. Feature store: Redis stores pre-computed user features (30/60/90-day spending profile, device fingerprint trust score, velocity counters). Tier 2 (async, minutes-to-hours): deep ML model (neural network on transaction graph, 500+ features). Reviews flagged cases, updates the Tier 1 model's training data. Storage: raw transactions in Cassandra (hot data 90 days, S3 for historical). Label store: confirmed fraud/not-fraud labels → model retraining weekly. Monitoring: PSI on all 150 features daily, AUC on labelled hold-out weekly.

Real-World Example:

PayPal's fraud system processes 450M+ transactions per month with 99.9%+ uptime requirements. Their two-tier architecture: Tier 1 (50ms SLA) uses feature store in Redis (pre-computed user behaviour features updated every 5 minutes by Flink) + XGBoost model served via ONNX on CPU. Tier 2 uses graph neural networks to detect fraud rings (multiple accounts coordinating). Key challenge: new card fraud patterns emerge daily — PayPal's model is retrained weekly on the most recent 90 days of labelled data, and new rule patterns are deployed within hours of detection. The false positive rate (0.3% of legitimate transactions blocked) costs PayPal \$150M/year in customer service calls and churn — carefully balanced against \$4B/year in fraud prevented. Every 1% improvement in recall (catching more fraud) saves PayPal \$40M at constant false positive rate.

Q5. Design a global chat system like WhatsApp.

Scale: 2B users, 100B messages/day. Architecture: Connection service: users connect via long-lived WebSocket connections to connection servers (geographically distributed). Online presence service (Redis pub/sub): tracks which connection server each user is on. Message routing: sender → sender's connection server → message service → Kafka → recipient's connection server → delivery via WebSocket. If recipient is offline: store in message queue DB (Cassandra, keyed by recipient). Message storage: Cassandra (messages table: conversation_id, message_id timestamp, sender_id, content_encrypted). End-to-end encryption: messages encrypted with Signal Protocol on device before sending. Server never decrypts content. Group messaging: group membership in Cassandra. Fan-out: for each group message, the messaging service writes once to Kafka; Kafka consumer fan-outs delivery to each member's connection server. Media: images/video uploaded to S3 first; only the S3 URL sent as a message.

Real-World Example:

WhatsApp serves 2B users with only 50 engineers — one of the most efficient engineering organisations in tech. Key design choices: long-lived WebSocket connections (XMPP protocol) instead of polling — dramatically reduces battery drain on mobile. Each server handles 1M+ concurrent WebSocket connections using Erlang's actor model (lightweight processes, message-passing, efficient I/O multiplexing). Message persistence: messages stored for 30 days if undelivered; deleted from server after delivery confirmation from client. WhatsApp's end-to-end encryption (Signal Protocol): each message is encrypted with a unique per-message key derived from a ratcheting protocol — even if WhatsApp's servers were compromised, messages cannot be decrypted. Group messages up to 1,024 members: sender's device derives per-recipient encrypted copies (client-side fan-out) rather than server-side fan-out, preserving E2E encryption.

Q6. Design a recommendation engine for Netflix.

Architecture: Offline (batch, nightly): collaborative filtering on the user-item interaction matrix (100M users × 15K titles). Matrix factorisation (ALS) produces user embedding vectors (U, 100-dimensional) and item embedding vectors (V, 100-dimensional). Stored in feature store. Candidate generation: for each user, retrieve top-1000 candidate titles using approximate nearest neighbour (HNSW) on U and V vectors. Online (real-time, <100ms): retrieve user embedding from feature store, retrieve top-1000 candidate IDs from ANN index. Ranking model (gradient boosting, 200 features): features include user-item embedding similarity, title novelty, time-of-day preferences, session context, recency decay. Returns re-ranked top-50. AB testing: every recommendation change runs for 2 weeks with 1M user treatment group. Primary metric: weekly viewing hours per user. Diversity controls: prevent the same genre/actor from dominating recommendations.

Real-World Example:

Netflix's recommendation system directly drives 80% of viewer activity (80% of hours watched are from recommendations, not search). Their most impactful model: collaborative filtering captures 'users like you watched X.' Content-based filtering captures 'you liked thrillers before, here's another thriller.' The hybrid is called 'Netflix Personalization': at scale, a gradient boosting model takes embeddings from both as features. When Netflix personalises artwork (showing a different thumbnail per user), a contextual bandit model (Thompson Sampling) continuously tests which thumbnail drives the most play for each user segment. Every UI element (row order, row titles, thumbnail art) is personalised — Netflix estimates this personalisation drives \$1B/year in retained subscriptions by keeping users engaged rather than cancelling.

Q7. Design a real-time collaborative document editor like Google Docs.

Core challenge: concurrent edits from multiple users must converge to the same document state. Algorithm: Operational Transformation (OT) or Conflict-free Replicated Data Types (CRDTs). OT approach: each edit is an operation (Insert(pos, char) or Delete(pos)). When two operations are concurrent, transform them against each other to maintain intent: if user A inserts at position 5 and user B deletes at position 3, A's operation must be transformed to Insert(4, char) because B's deletion shifted positions. Architecture: each document has a dedicated server (operation server). Clients send operations + last seen revision to the operation server. Server serialises all concurrent operations using OT, broadcasts the transformed operations to all clients, increments revision number. Client applies received operations immediately (optimistic concurrency). Presence: WebSocket connection per user, cursor positions broadcasted to room. Persistence: operations log in Spanner (strongly consistent, globally replicated). Periodically compacted into full document snapshots.

Real-World Example:

Google Docs uses OT as described above, built on the Google Wave OT implementation. The operation server serialises all concurrent edits — at Google scale, a popular document may have 100 simultaneous editors. Each operation is applied in <10ms by the server; clients see their own edits immediately (optimistic) and others' edits within 100-200ms. The engineering challenge: OT is notoriously hard to implement correctly — Google spent years debugging edge cases. Figma switched from OT to CRDTs (specifically, a Last-Write-Wins Register per document element) because CRDTs don't require a central server to serialise operations — beneficial for Figma's peer-to-peer real-time design collaboration. Both approaches achieve eventual consistency for collaborative editing; OT provides stronger intent-preservation guarantees while CRDTs offer simpler distributed implementation.

Q8. Design a distributed search engine.

Architecture: Crawler (distributed BFS) → Kafka → Parser/Indexer → Inverted Index (distributed across shards) → Query Processor → Ranking → API. Inverted index: for each term, store a posting list: (doc_id, term_frequency, position_list). Sharding: partition posting lists by term hash across N index shards. Query processing: query → parse → [term1, term2, term3] → scatter query to all shards → each shard returns doc IDs matching the terms → gather merge → TF-IDF or BM25 ranking → return top-K results. Ranking: $BM25\ score = TF \times IDF \times \frac{(k_1+1)}{(TF+k_1) \times (1-b+b \times doc_len/avg_doc_len)}$. Distributed: each shard computes local scores independently; coordinator merges global top-K. Caching: query result cache (Redis, LRU) for common queries. Document store: raw document text in HDFS/GCS; forward index (doc_id → full text) for snippet generation.

Real-World Example:

Google's search infrastructure processes 8.5B queries/day. Their inverted index spans thousands of servers globally, sharded by term. For a 3-term query, Google routes to shards for each term simultaneously, collects doc IDs, computes intersection (Biggie merge), then scores with a PageRank-weighted BM25 variant. Query result caching: common queries ('weather', 'news') are served from cache — 30%+ hit rate. The serving infrastructure must return results in <200ms globally. Google's Colossus (GFS successor) stores petabytes of inverted index data. Real-time indexing: news and trending content is indexed within seconds of publication via a separate 'freshness index' that is merged with the main index at query time. The main index (web crawl-based) is updated every few weeks; the freshness index updates continuously.

Q9. Design a system handling 1 million users watching a live sports event simultaneously.

Challenge: single live stream delivered to 1M concurrent viewers in real time with <10 second latency. Architecture: Ingest: broadcast source → transcoding servers (encode to 6 quality tiers in real time, HLS/MPEG-DASH). Each quality tier segmented into 2-second chunks. Origin: encoded segments pushed to origin storage (S3). CDN: 1M concurrent viewers × 5 Mbps average = 5 Tbps total bandwidth. No single CDN PoP can handle this — distribute across 1,000+ PoPs worldwide. CDN pre-loading: CDN origin-pull each segment immediately after creation. By the time viewers request the segment, it's already on the CDN edge node. Manifest file: each client fetches the m3u8 manifest every 2 seconds. Manifest contains URLs for the latest segments. Client ABR (Adaptive Bitrate) algorithm: adjusts quality tier based on measured download speed. Live latency: standard HLS has 30s delay (3 segments × 10s). Low-latency HLS (LLHLS) reduces to 5-8 seconds using 0.5-second segments with partial segment delivery.

Real-World Example:

During the Super Bowl (100M+ US viewers, 50M streaming), AWS MediaLive + CloudFront handles the transcoding and delivery for major broadcasters. The CDN pre-warms for the Super Bowl by pre-positioning edge servers globally 6 hours before kickoff. CloudFront's 600+ PoPs handle the burst without any single bottleneck. Key challenge: the segment request pattern is perfectly synchronised — all 50M viewers request the same segment at the same second. CDN caching is 100% effective (every viewer gets the same segment). Without CDN, each viewer would pull directly from origin — $50M \times 5 \text{ Mbps} = 250 \text{ Tbps}$ from a single origin, impossible. The CDN transforms 250 Tbps of origin demand into ~250 Gbps (1,000× reduction via cache hits). Actual origin bandwidth: only uncached segments (new segments every 2 seconds) are fetched from origin.

Q10. Design a hotel booking system like Booking.com.

Core entities: Hotels, Rooms, Bookings, Users. Key challenge: prevent double-booking of the same room. Schema: rooms (room_id, hotel_id, type, capacity, base_price), room_inventory (room_id, date, total_rooms INT, booked_rooms INT), bookings (booking_id, user_id, room_id, check_in, check_out, status). Availability check: `SELECT rooms_available = total_rooms - booked_rooms FROM room_inventory WHERE room_id = X AND date BETWEEN check_in AND check_out`. If available on ALL dates: proceed to booking. Booking creation (prevent race condition): `UPDATE room_inventory SET booked_rooms = booked_rooms + 1 WHERE room_id = X AND date BETWEEN check_in AND check_out AND (total_rooms - booked_rooms) > 0`; — if this updates 0 rows: concurrency conflict, retry. This optimistic locking avoids SELECT FOR UPDATE locks. Pricing: dynamic pricing engine adjusts base price × (demand multiplier + seasonal multiplier + last-minute factor). Caching: room availability for the next 90 days cached in Redis (invalidated on booking). Search: Elasticsearch for hotel discovery (location, amenities, ratings).

Real-World Example:

Booking.com handles 1.5M room nights booked per day across 28M+ listings. Their room inventory system uses a per-room-per-date locking strategy: the `UPDATE ... WHERE booked < total` constraint ensures exactly correct availability without two-phase locking. During peak booking periods (summer travel season), a popular room in Paris receives thousands of concurrent availability checks. With optimistic locking: 99.9% succeed on the first attempt; 0.1% conflict and retry. Each retry takes <5ms. The alternative (`SELECT FOR UPDATE` with 5-10 second locks during payment processing) would serialise all bookings for the same room, creating a thundering-herd effect. Booking.com's system is designed around the insight that conflicts are rare — optimistic locking is faster than pessimistic locking in low-conflict scenarios.